

Using ASP.NET Core 6 Web API and NoSQL Databases

Understanding the Basics of APIs



Edward Curren

@edwardcurren

<http://www.edwardcurren.com>



Course Overview



Understanding APIs

- Designing the API
- A look at the finished project
- Getting the project started

The data and the database

- We're going to use MongoDB
- We'll be putting an adapter between the DB and the API code



Course Overview



Writing the API

- HTTP Verbs
- Routing
- Security
- Passing data
- Using HTTP return codes
- Auto-generated documentation

Connecting a web site



Module Overview



The basics of an API

Design factors

Endpoints

Structuring URIs

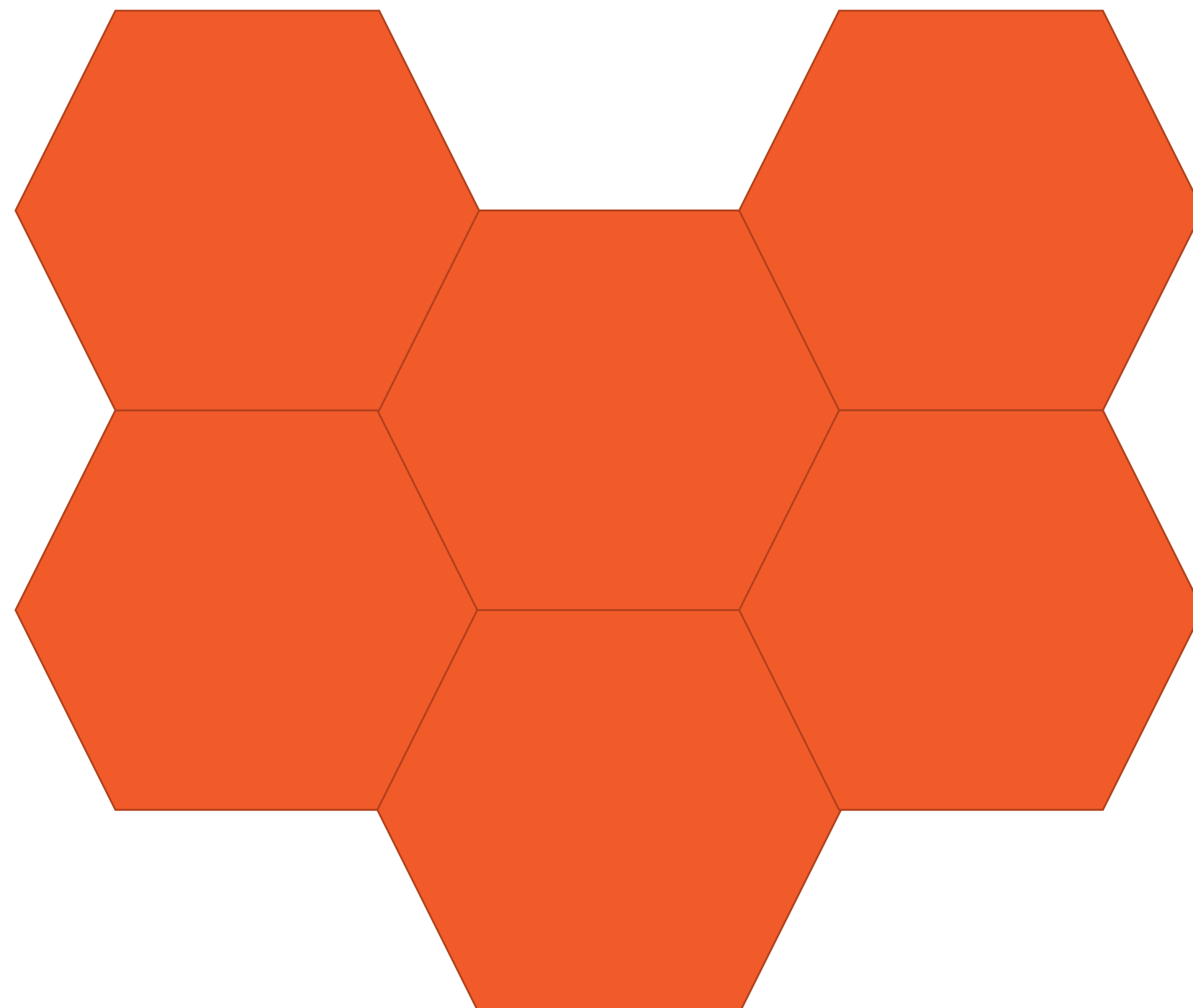
Security and versioning

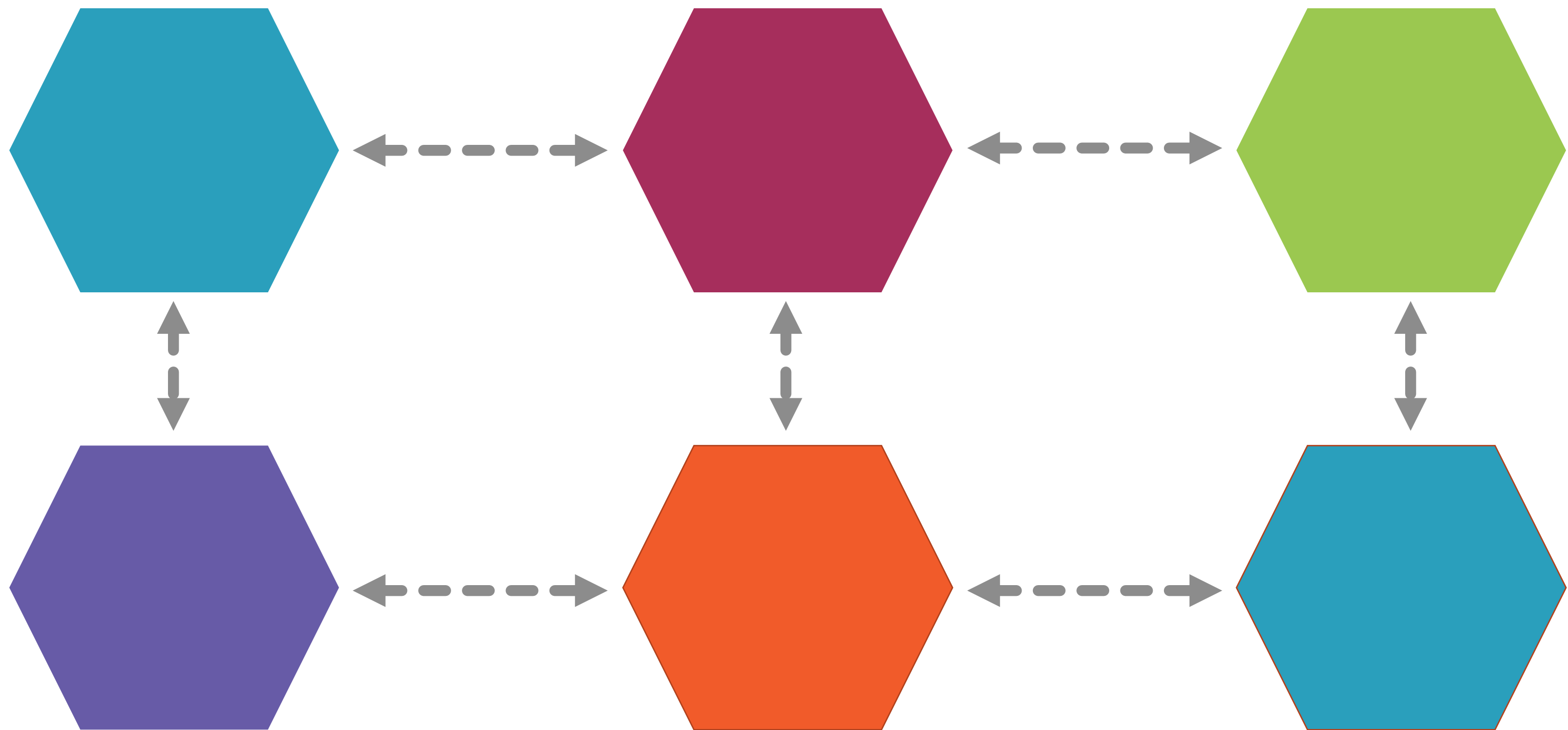
Get hands on



Why Use an API









SOAP vs. REST APIs

SOAP

SOAP is a protocol.

SOAP stands for Simple Object Access Protocol.

SOAP can't use REST because it is a protocol.

SOAP uses services interfaces to expose the business logic.

SOAP defines standards to be strictly followed.

SOAP requires more bandwidth and resource than REST.

REST

REST is an architectural style.

REST stands for REpresentational State Transfer.

REST can use SOAP web services because it is a concept and can use any protocol like HTTP, SOAP.

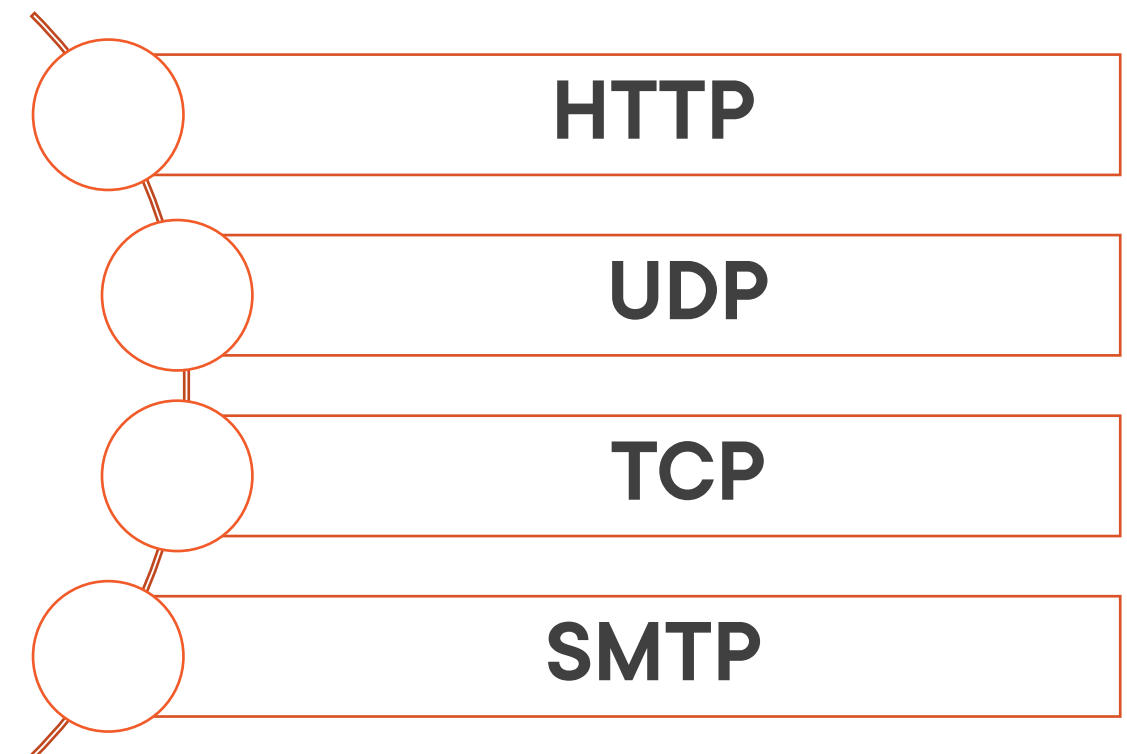
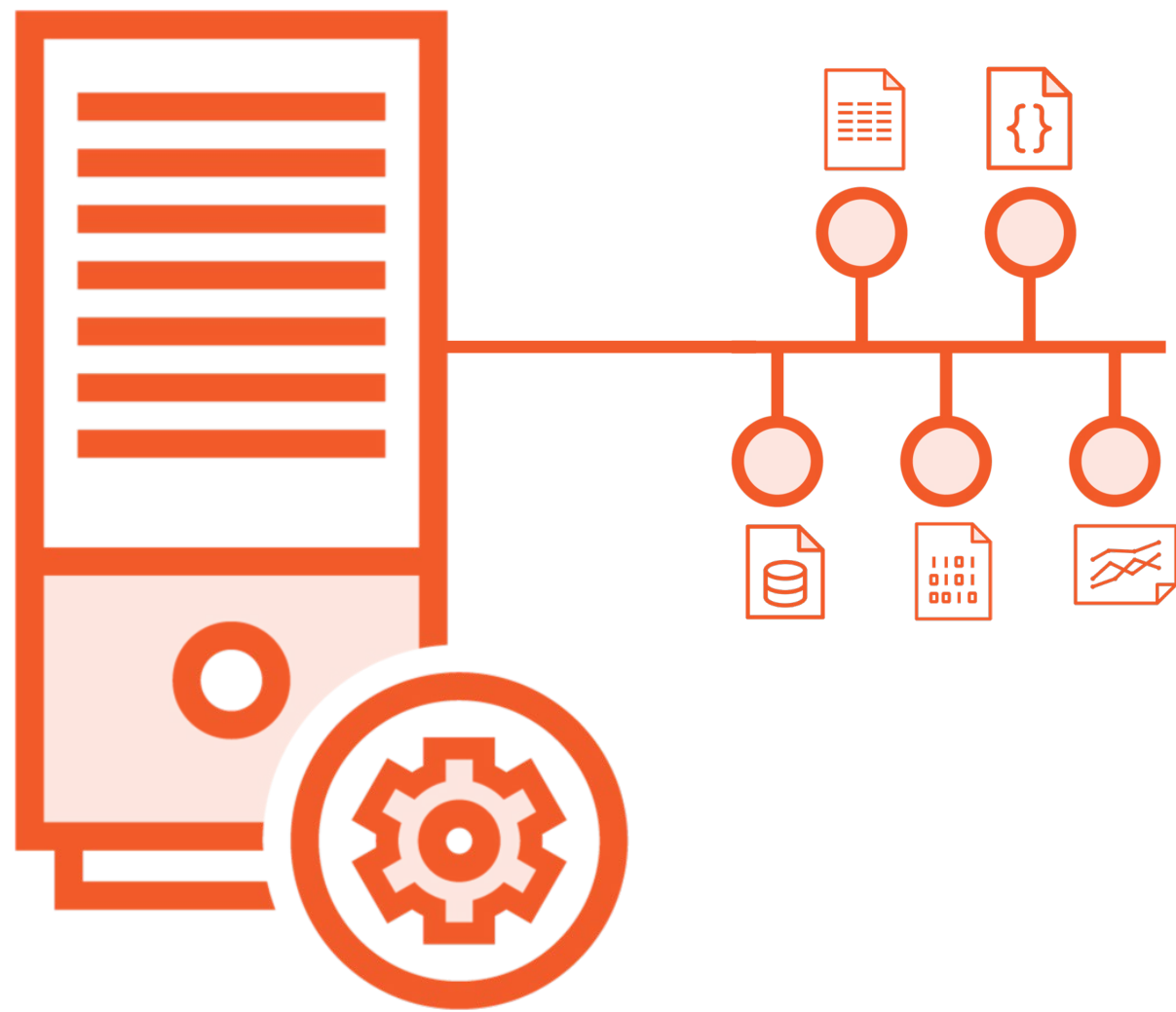
REST uses URI to expose business logic.

REST does not define too much standards like SOAP.

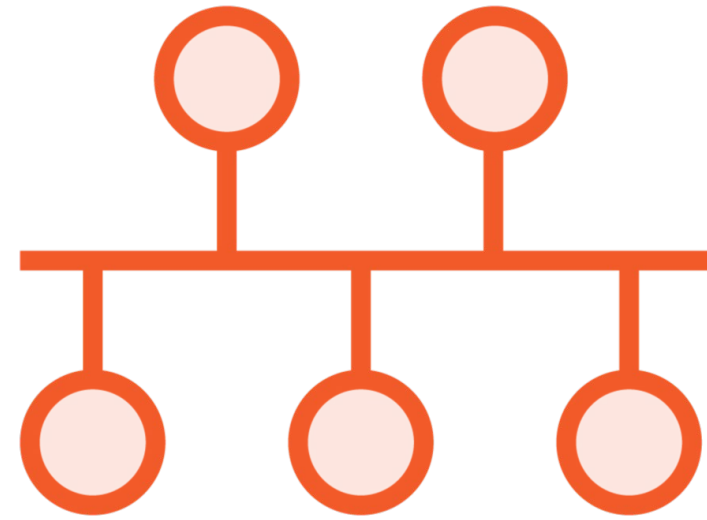
REST requires less bandwidth and resource than SOAP.



SOAP



Transition of Enterprise Applications



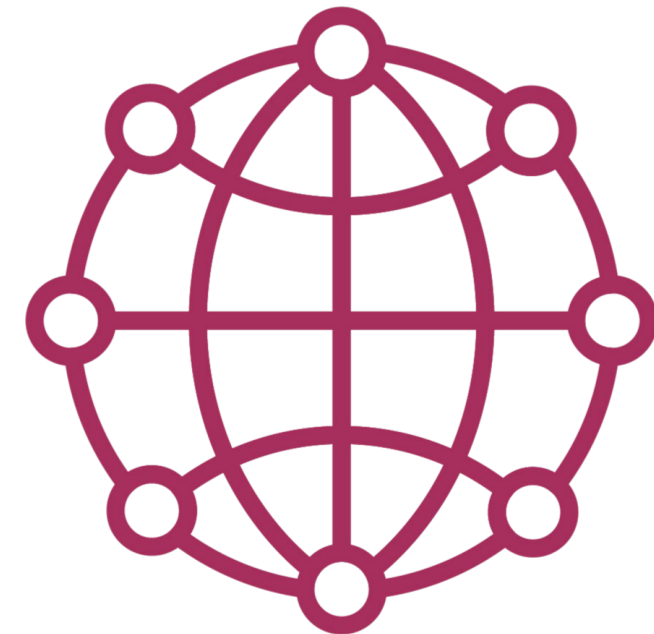
Closed Local Area
Network Applications
Deployed to On-Site
Local Desktops

← Pre-Internet

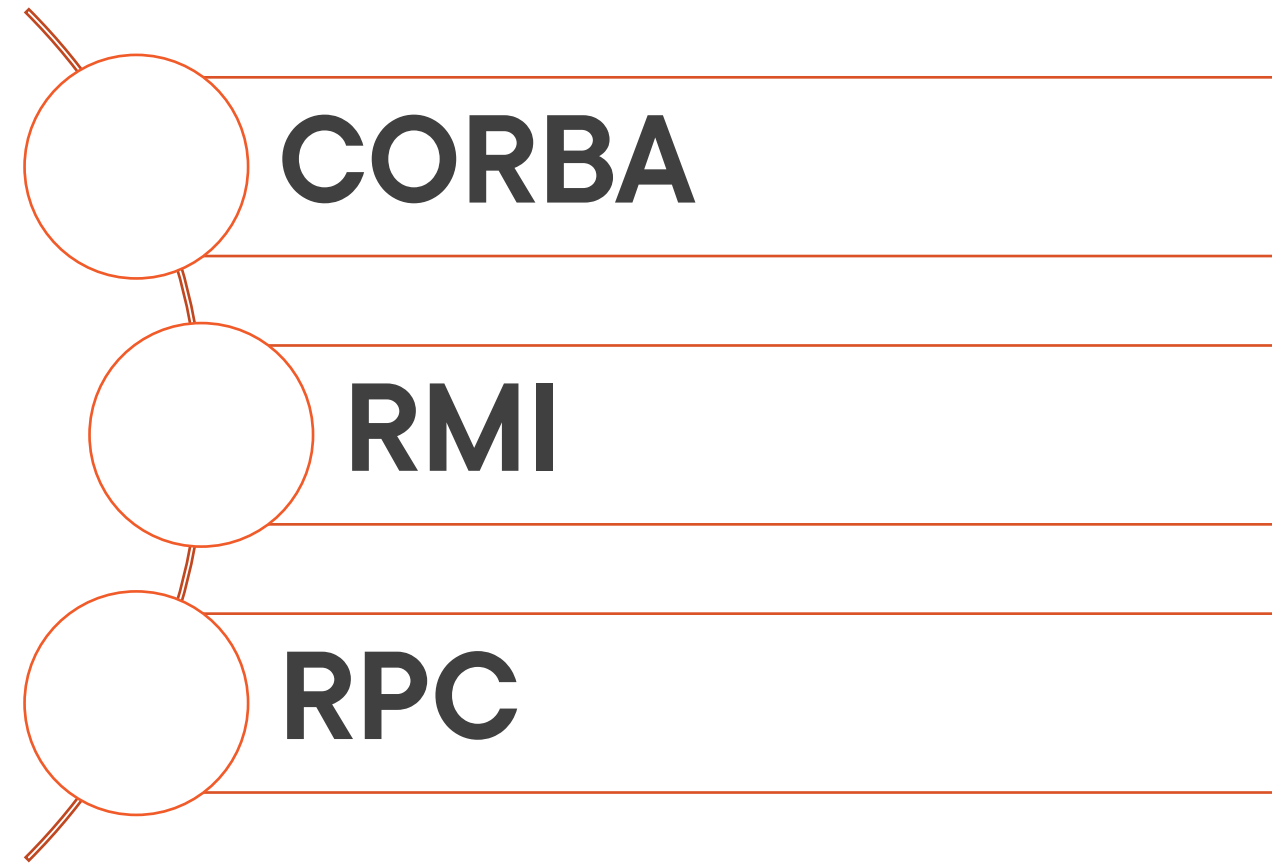


Post-Internet →

Large Scale Internet
Application Development
and Systems Integration



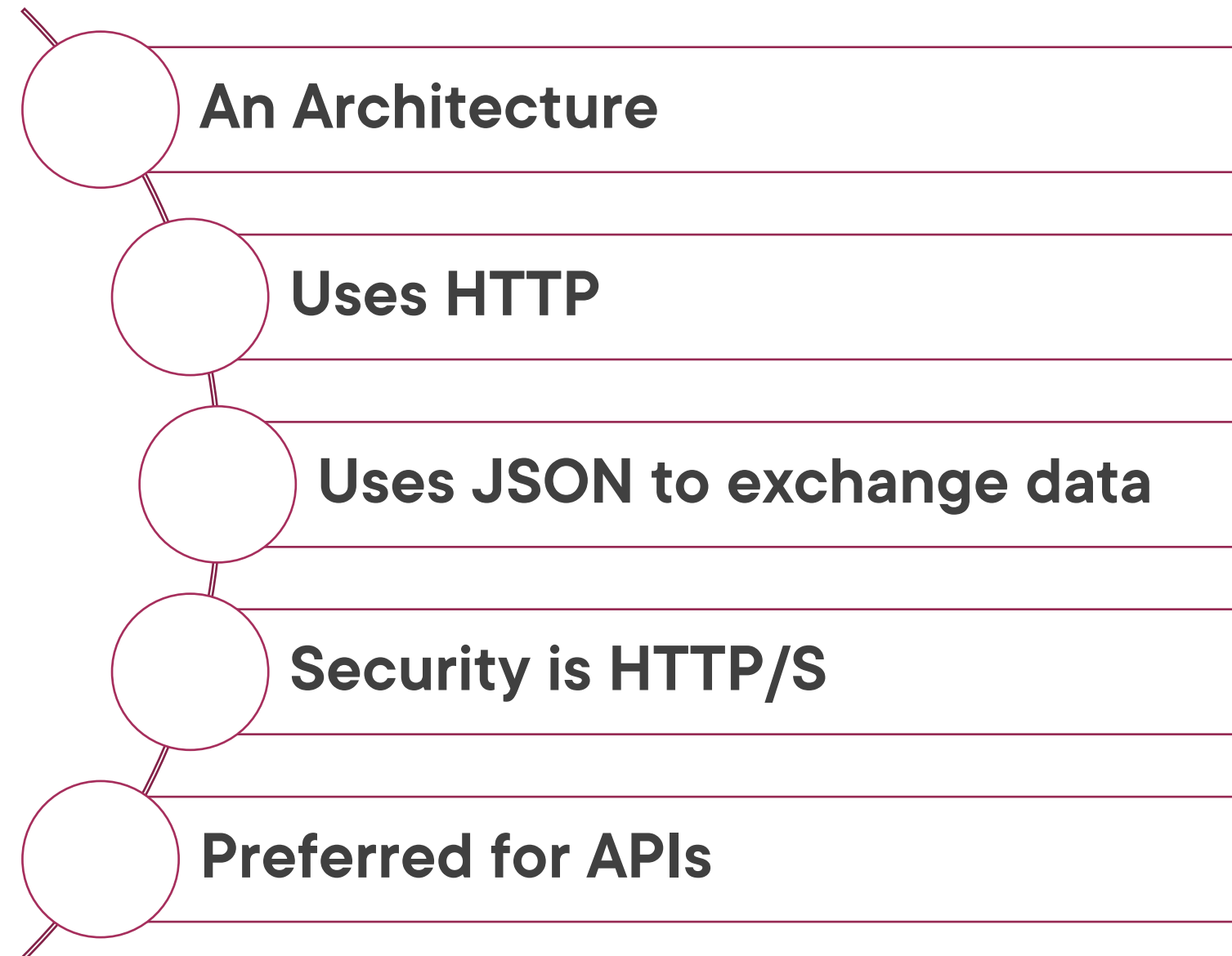
SOAP



SOAP vs REST



SOAP vs REST



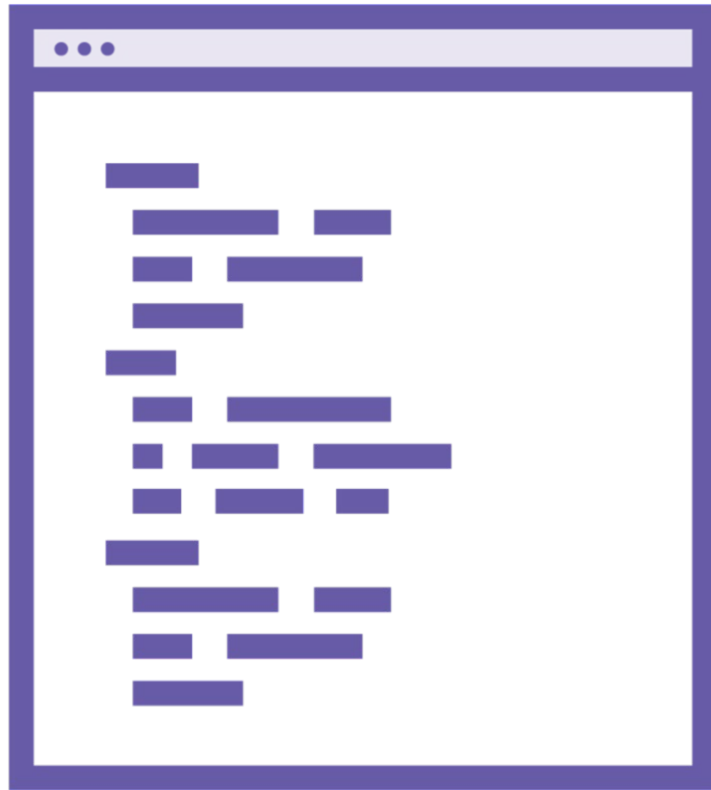
Up Next: Designing our REST API



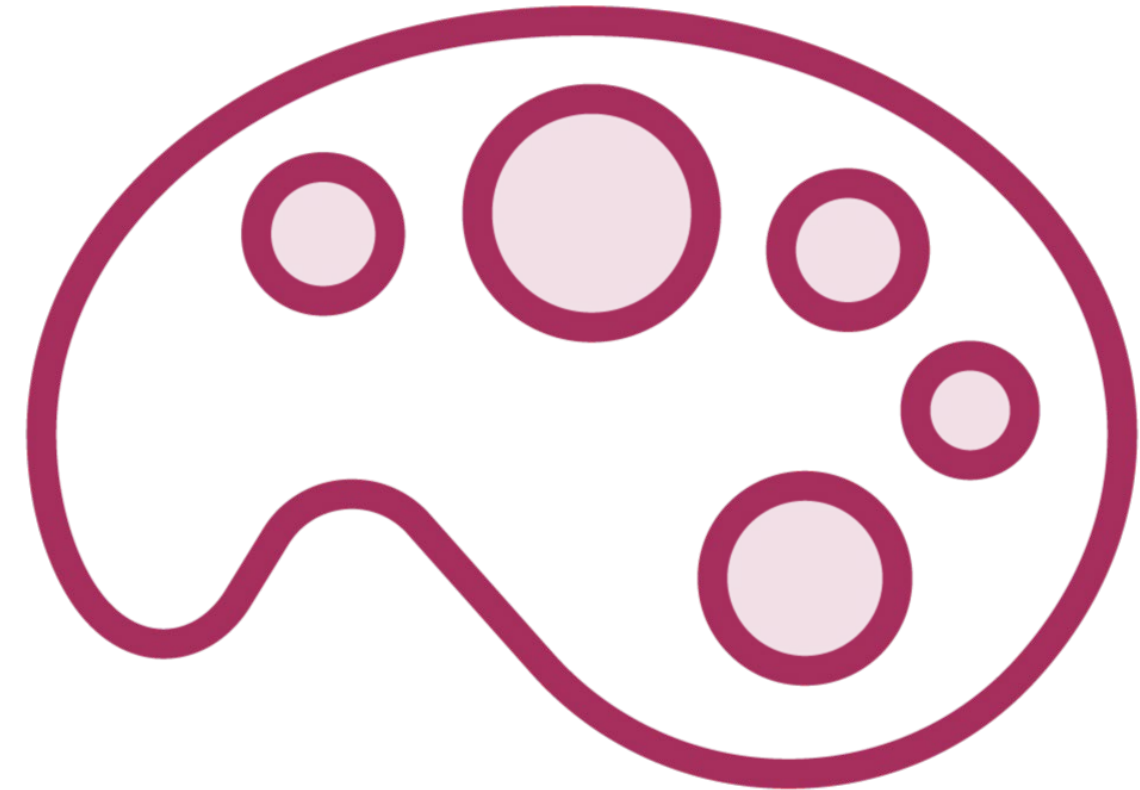
Designing our REST API



The Science and the Art of Software

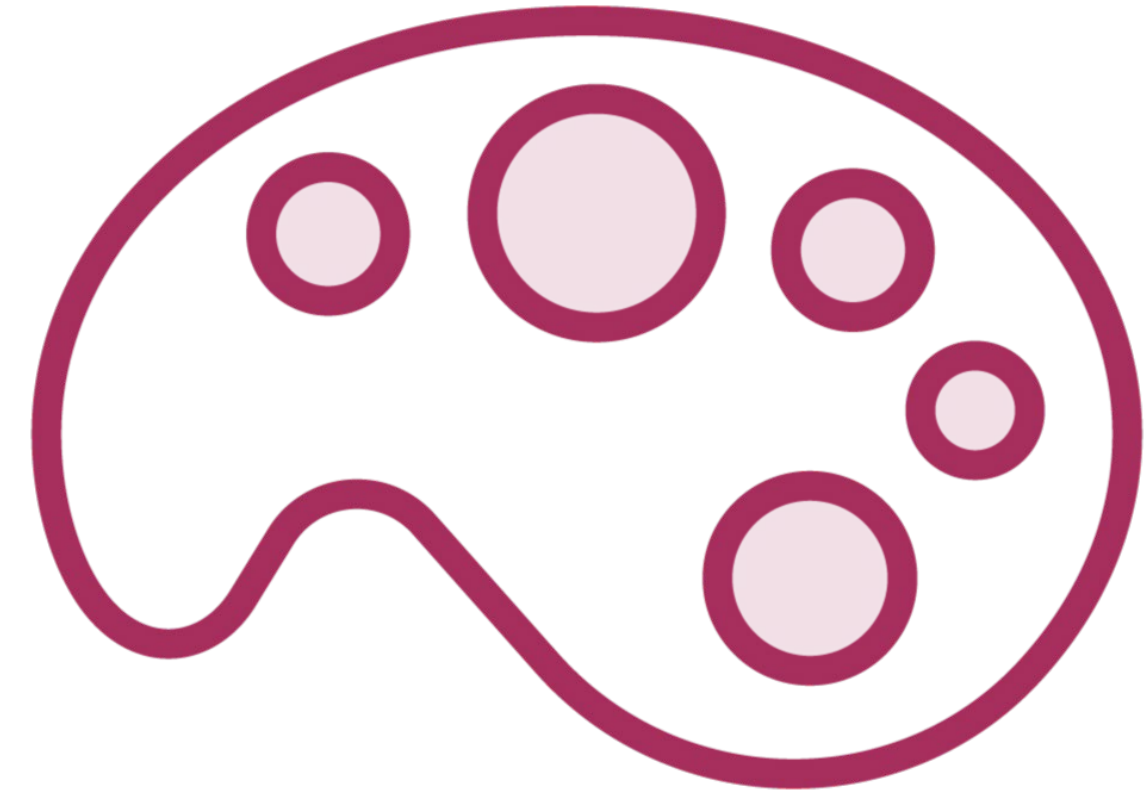


Code Syntax
Control Flow
Data Manipulation



Design Decisions
Architectural Considerations

The Science and the Art of Software



Design Decisions
Architectural Considerations





Deep dive into designing APIs

Designing RESTful Web APIs

Shawn Wildermuth





Rules for designing a good API

REST API Design Rulebook

Mark Masse

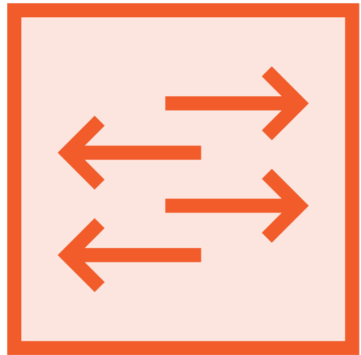


Flight Plan Data Model

string	FlightPlanId	// Unique Flight Plan Identifier
string	AircraftIdentification	// The tail number of the Aircraft
string	AircraftType	// Make and Model of the Aircraft
int	Airspeed	// The expected airspeed of the flight
int	Altitude	// The altitude the aircraft will maintain
string	FlightType	// Will this be an IFR or a VFR flight
int	FuelHours	// The amount of fuel onboard in hours
int	FuelMinutes	// The minutes of fuel beyond the hour mark
DateTime	DepartureTime	// The time the flight will depart
DateTime	ArrivalTime	// The expected time the flight will arrive
string	DepartureAirport	// The airport from which the flight is departing
string	ArrivalAirport	// The airport at which the flight is arriving
string	Route	// The route the flight will travel
string	Remarks	// Any extra information relevant to the flight
int	NumberOnBoard	// The number of people onboard the flight



Important Considerations for API Designs



The patterns, formats and interfaces implemented for the API are consistent across the API.



We can rely on the API to provide a stable interface indefinitely.



With a basic knowledge of the resource, we can predict the URI endpoint structure.



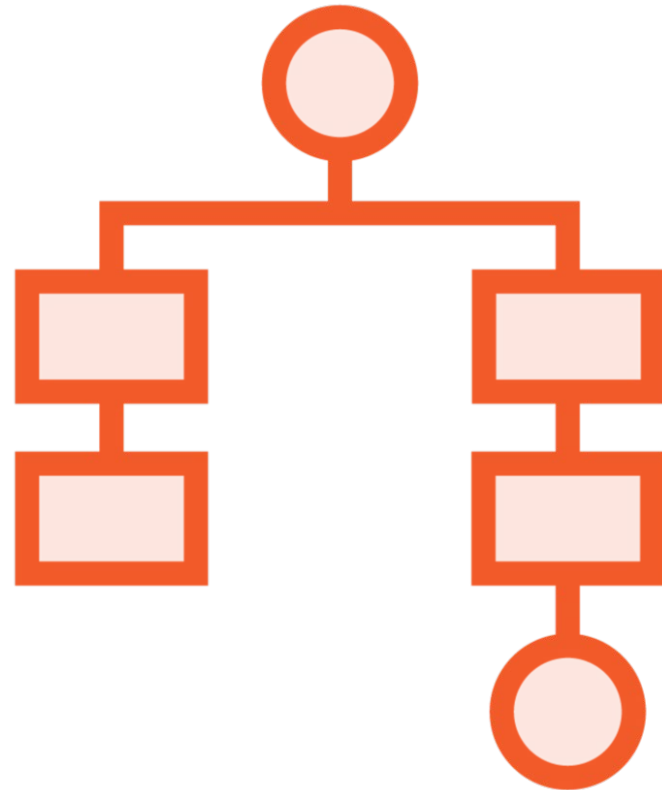
URI vs. URL

URI

URL



The Science and the Art of Software



**Simple Framework to
Produce Good API Designs**

Designing for humans and machines

Needs to clearly communicate the
resource model.



Next Up: Endpoints



Endpoints



Flight Plan API Endpoints

	HTTP VERB	URI Path	Returns Flight Plan	Requires Flight Plan	Requires Flight Plan Id
	GET	http://...			
	GET	http://.../{flight plan id}			



Flight Plan API Endpoints

	HTTP VERB	URI Path	Returns Flight Plan	Requires Flight Plan	Requires Flight Plan Id
	GET	http://.../{flight plan id}			
	PUT	http://.../{flight plan id}			
	DELETE	http://.../{flight plan id}			



Flight Plan API Endpoints

	HTTP VERB	URI Path
	GET	http://...
	GET	http://.../d165c40d77244b518617dd0e70b4cc6d
	POST	http://.../file
	PUT	http://.../d165c40d77244b518617dd0e70b4cc6d
	DELETE	http://.../d165c40d77244b518617dd0e70b4cc6d



Flight Plan API Endpoints

	HTTP VERB	URI Path	Returns Flight Plan	Requires Flight Plan	Requires Flight Plan Id
	GET	http://...			
	GET	http://.../{flight plan id}			
	POST	http://.../file			
	PUT	http://.../{flight plan id}			
	DELETE	http://.../{flight plan id}			



Flight Plan API Endpoints

	HTTP VERB	URI Path
	GET	http://api.mydomain.com:3000
	GET	http://api.mydomain.com:3000/ {flight plan id}
	POST	http://api.mydomain.com:3000/file
	PUT	http://api.mydomain.com:3000/ {flight plan id}
	DELETE	http://api.mydomain.com:3000/ {flight plan id}



Flight Plan API Endpoints

HTTP VERB URI Path



GET

<http://dev.api.mydomain.com:3000>



GET

<http://dev.api.mydomain.com:3000/{flight plan id}>



POST

<http://dev.api.mydomain.com:3000/file>



PUT

<http://dev.api.mydomain.com:3000/{flight plan id}>



DELETE

<http://dev.api.mydomain.com:3000/{flight plan id}>



Flight Plan API Endpoints

HTTP VERB URI Path



GET

http://qa.api.mydomain.com:3000



GET

http://qa.api.mydomain.com:3000/{flight plan id}



POST

http://qa.api.mydomain.com:3000/file



PUT

http://qa.api.mydomain.com:3000/{flight plan id}



DELETE

http://qa.api.mydomain.com:3000/{flight plan id}



Flight Plan API Endpoints

HTTP VERB URI Path



GET

<http://localhost:3000/api>



GET

<http://localhost:3000/api/{flight plan id}>



POST

<http://localhost:3000/api/file>



PUT

<http://localhost:3000/api/{flight plan id}>



DELETE

<http://localhost:3000/api/{flight plan id}>

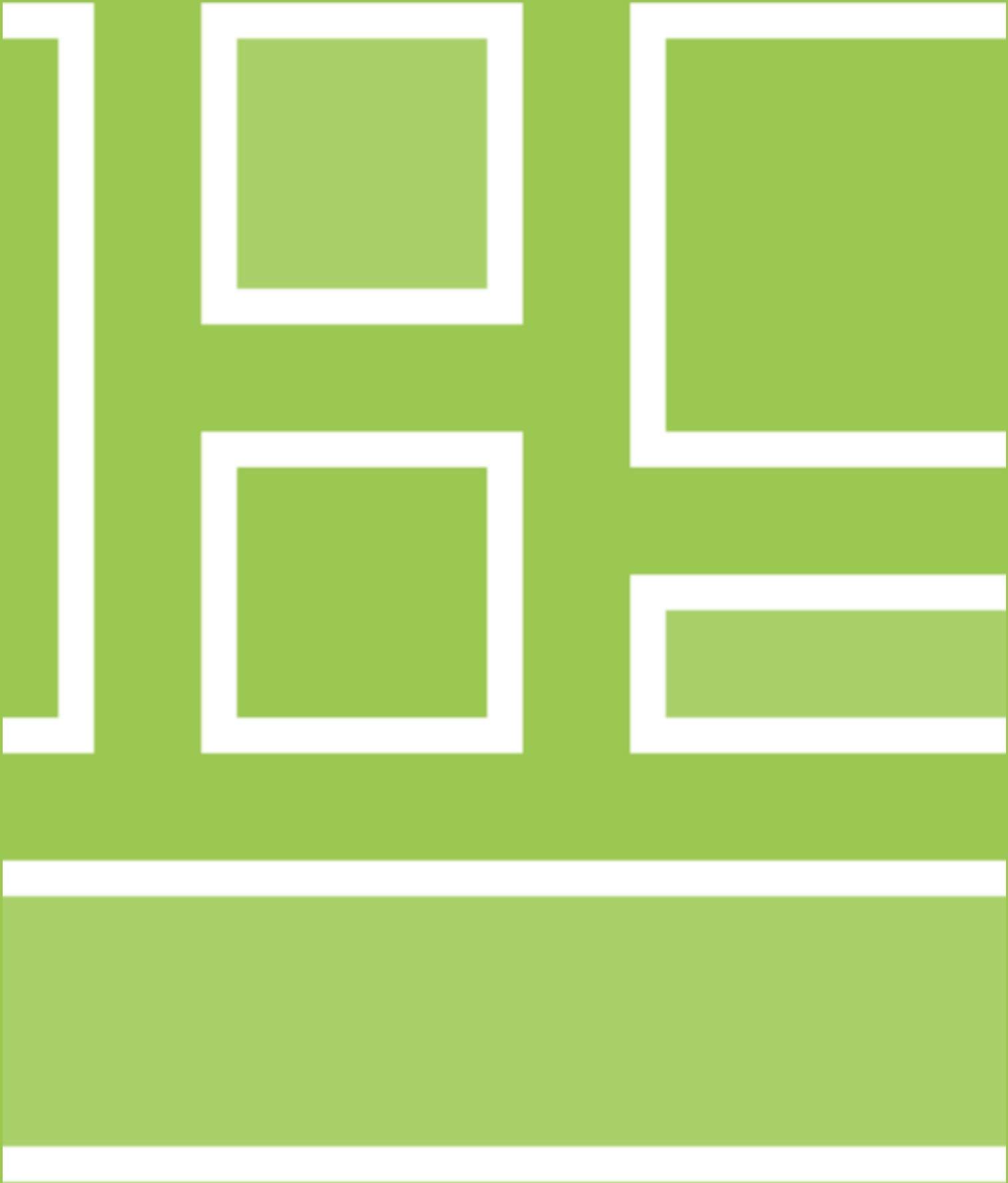


Next Up: Structuring URIs



Structuring URIs





API Resource

The entity manipulated by API endpoints.





<https://api.domain.com/v1/drone/takeoff>

<https://api.domain.com/v1/drone/land>

<https://api.domain.com/v1/drone/climb/{altitude}>

<https://api.domain.com/v1/drone/descend/{altitude}>

<https://api.domain.com/v1/drone/move/{latitude}/{longitude}>

<https://api.domain.com/v1/drone/pictures/snap>

<https://api.domain.com/v1/drone/video/start>

<https://api.domain.com/v1/drone/video/stop>

Flight Plan API Endpoints

	HTTP VERB	URI Path	Returns Flight Plan	Requires Flight Plan	Requires Flight Plan Id
	GET	http://...			
	GET	http://.../{flight plan id}			
	POST	http://.../file			
	PUT	http://.../{flight plan id}			
	DELETE	http://.../{flight plan id}			



Flight Plan API Endpoints

HTTP VERB URI Path



GET

<http://.../{flight plan id}/airports/departure>



GET

<http://.../{flight plan id}/route>



GET

<http://.../{flight plan id}/times/enroute>



Flight Plan Data Model

string	FlightPlanId	// Unique Flight Plan Identifier
string	AircraftIdentification	// The tail number of the Aircraft
string	AircraftType	// Make and Model of the Aircraft
int	Airspeed	// The expected airspeed of the flight
int	Altitude	// The altitude the aircraft will maintain
string	FlightType	// Will this be an IFR or a VFR flight
int	FuelHours	// The amount of fuel onboard in hours
int	FuelMinutes	// The minutes of fuel beyond the hour mark
DateTime	DepartureTime	// The time the flight will depart
DateTime	ArrivalTime	// The expected time the flight will arrive
string	DepartureAirport	// The airport from which the flight is departing
string	ArrivalAirport	// The airport at which the flight is arriving
string	Route	// The route the flight will travel
string	Remarks	// Any extra information relevant to the flight
int	NumberOnBoard	// The number of people onboard the flight



Flight Plan API Endpoints

HTTP VERB URI Path



GET

<http://.../{flight plan id}/airports/departure>



GET

<http://.../{flight plan id}/route>



GET

<http://.../{flight plan id}/times/enroute>



Flight Plan Data Model

string	FlightPlanId	// Unique Flight Plan Identifier
string	AircraftIdentification	// The tail number of the Aircraft
string	AircraftType	// Make and Model of the Aircraft
int	Airspeed	// The expected airspeed of the flight
int	Altitude	// The altitude the aircraft will maintain
string	FlightType	// Will this be an IFR or a VFR flight
int	FuelHours	// The amount of fuel onboard in hours
int	FuelMinutes	// The minutes of fuel beyond the hour mark
DateTime	DepartureTime	// The time the flight will depart
DateTime	ArrivalTime	// The expected time the flight will arrive
string	DepartureAirport	// The airport from which the flight is departing
string	ArrivalAirport	// The airport at which the flight is arriving
string	Route	// The route the flight will travel
string	Remarks	// Any extra information relevant to the flight
int	NumberOnBoard	// The number of people onboard the flight



Flight Plan API Endpoints

HTTP VERB URI Path



GET

<http://.../{flight plan id}/airports/departure>



GET

<http://.../{flight plan id}/route>



GET

<http://.../{flight plan id}/times/enroute>



Flight Plan API Endpoints

	HTTP VERB	URI Path	Returns Flight Plan	Requires Flight Plan	Requires Flight Plan Id
	GET	http://...			
	GET	http://.../{flight plan id}			
	POST	http://.../file			
	PUT	http://.../{flight plan id}			
	DELETE	http://.../{flight plan id}			



HTTP Response Codes



<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>





418 I'm a teapot

The HTTP `418 I'm a teapot` client error response code indicates that the server refuses to brew coffee because it is, permanently, a teapot. A combined coffee/tea pot that is temporarily out of coffee should instead return 503. This error is a reference to Hyper Text Coffee Pot Control Protocol defined in April Fools' jokes in 1998 and 2014.

Some websites use this response for requests they do not wish to handle, such as automated queries.



Common HTTP Status Codes

200

Ok – No problems processing the request

400

Bad Request – The server can't process the request

401

Unauthorized – The method being called requires authentication

404

Not Found – The server doesn't have the info that's being asked for

500

Internal Server Error – Something went wrong in the server itself.



Flight Plan API Endpoints

	HTTP VERB	URI Path	Possible Status Codes		
	GET	http://...	204 (No Content)		
	GET	http://.../{flight plan id}	400 (Bad Request)		
	POST	http://.../file	400 (Bad Request)	500 (Internal Server Error)	
	PUT	http://.../{flight plan id}	400 (Bad Request)	404 (Not Found)	500 (Internal Server Error)
	DELETE	http://.../{flight plan id}	404 (Not Found)		

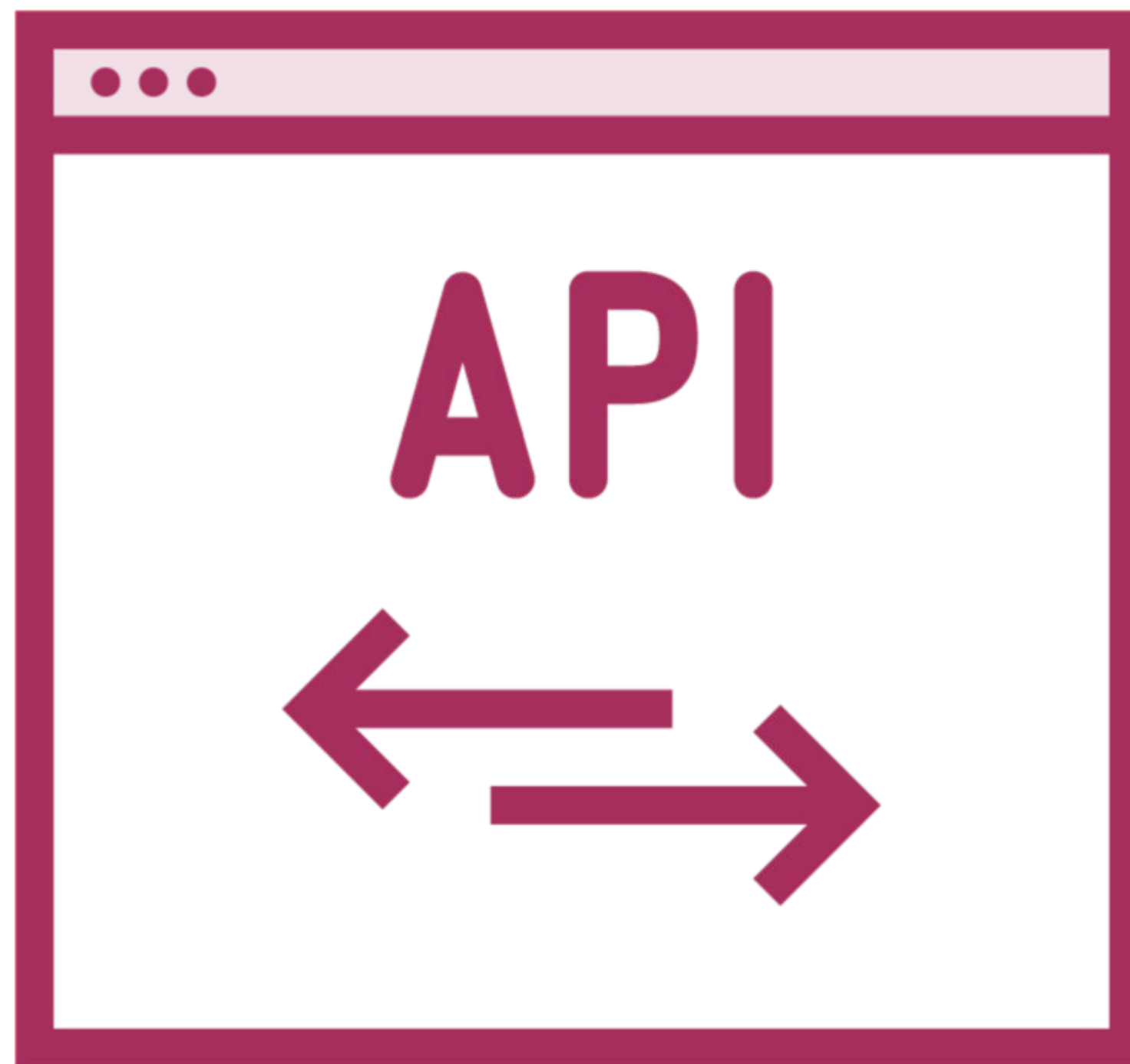


Up Next: Security and Versioning



Security and Versioning





Types of Authentication



OAuth 2.0



OpenID Connect



Basic Authentication



Active Directory





Overview of OAuth 2.0

Getting Started with OAuth 2.0

Scott Brady





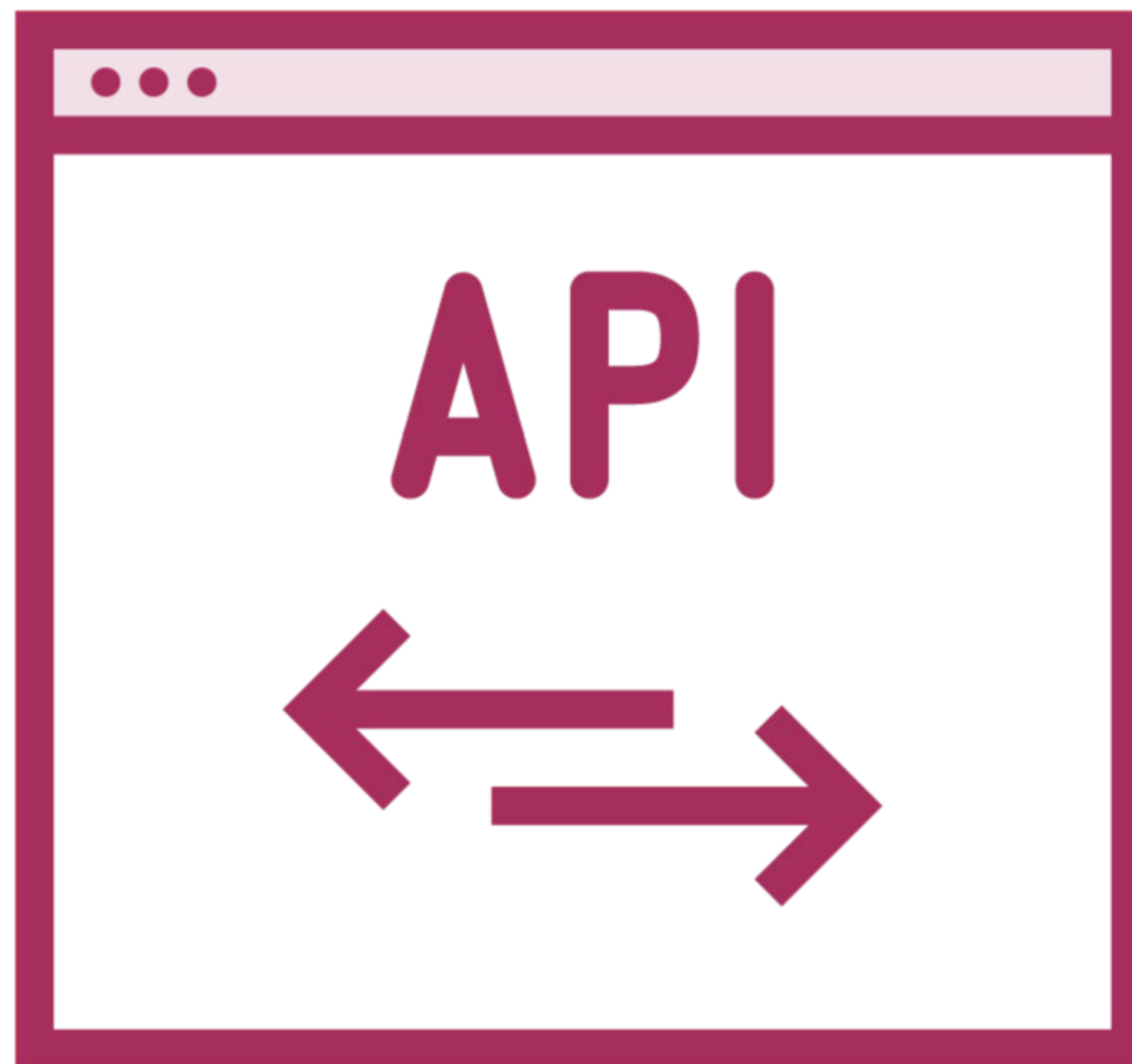
OAuth and ASP.NET

Securing ASP.NET Core 3 with OAuth2 and OpenID Connect

Kevin Dockx



API Versioning



When Do I Change an API Version?

Add

```
https://api.domain.com/v1/flightplans/activate
```

Remove

```
https://api.domain.com/v1/flightplans/close
```

Before

```
https://api.domain.com/v1/flightplans/{flightplanid}
```

After

```
https://api.domain.com/v1/flightplans/{datefiled}
```



When Do I Change an API Version?

```
{
  flight_plan_id string
  aircraft_identification string
  aircraft_type string
  airspeed integer($int32)
  altitude integer($int32)
  flight_type string
  fuel_hours integer($int32)
  fuel_minutes integer($int32)
  departure_time string($date-time)
  estimated_arrival_time string($date-time)
  departing_airport string
  arrival_airport string
  route string
  remarks string
  number_onboard integer($int32)
}
```

```
{
  flight_plan_id string
  aircraft_identification string
  aircraft_type string
  airspeed integer($int32)
  altitude integer($int32)
  flight_type string
  fuel_hours integer($float)
  fuel_minutes integer($float)
  departure_time string($date-time)
  time_enroute string($timespan)
  departing_airport string
  arrival_airport string
  route string
  remarks string
  number_onboard integer($int32)
}
```



When Do I Change an API Version?

```
{  
  flight_plan_id string  
  aircraft_identification string  
  aircraft_type string  
  airspeed integer($int32)  
  altitude integer($int32)  
  flight_type string  
  fuel_hours integer($int32)  
  fuel_minutes integer($int32)  
  departure_time string($date-time)  
  estimated_arrival_time string($date-time)  
  departing_airport string  
  arrival_airport string  
  route string  
  remarks string  
  number_onboard integer($int32)  
}
```

```
{  
  flight_plan_id string  
  aircraft_identification string  
  aircraft_type string  
  airspeed integer($int32)  
  altitude integer($int32)  
  flight_type string  
  fuel_hours integer($float)  
  fuel_minutes integer($float)  
  departure_time string($date-time)  
  time_enroute string($timespan)  
  departing_airport string  
  arrival_airport string  
  route string  
  remarks string  
  number_onboard integer($int32)  
}
```



API Versioning



`https://.../api/v1/...`



Version = 3.6 (Major = 3 Minor = 6)



`https://.../api/whatever?version=3.6`



`'accept-version': 3.6`



Up Next: Project Demo



Project Demo



Up Next: Setting up the Development Environment



Setting up the Development Environment



Up Next: Starting the Project



Starting the Project





Startup.cs

Had the ConfigureServices() method (to configure the services) and the Configure() method to configure the HTTP pipeline.



Same Origin Policy

The same-origin policy is a critical security mechanism that restricts how a document or script loaded by one origin can interact with a resource from another origin.

It helps isolate potentially malicious documents, reducing possible attack vectors. For example, it prevents a malicious website on the Internet from running JS in a browser to read data from a third-party webmail service (which the user is signed into) or a company intranet (which is protected from direct access by the attacker by not having a public IP address) and relaying that data to the attacker.

https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy



Origin

Two URLs have the same origin if the protocol, port (if specified), and host are the same for both. You may see this referenced as the "scheme/host/port tuple", or just "tuple". (A "tuple" is a set of items that together comprise a whole — a generic form for double/triple/quadruple/quintuple/etc.)



Cross-Origin Resource Sharing (CORS)

Cross-Origin Resource Sharing ([CORS](#)) is an [HTTP](#)-header based mechanism that allows a server to indicate any [origins](#) (domain, scheme, or port) other than its own from which a browser should permit loading resources. CORS also relies on a mechanism by which browsers make a "preflight" request to the server hosting the cross-origin resource, in order to check that the server will permit the actual request. In that preflight, the browser sends headers that indicate the HTTP method and headers that will be used in the actual request.



Up Next: Summary



Summary



Covered much of the “why’s” of API development

We will be hands on coding for the rest of the course



Next Module: The Data

